Université Nationale des Sciences, Technologies, Ingénierie et Mathématiques (UNSTIM) d'Abomey

École Nationale Supérieure de Génie Mathématique et Modélisation (ENSGMM)

Unité d'enseignement : Système d'Information

Modèle NoSQL orienté colonnes

Column-Family / Wide-Column

Groupe 4

AGRO Rachidi
ANATO Krisland
HOUNSOUNOU Marius

Sous la supervision de

Dr (MC). KONNON Abel

Année universitaire : 2026-2027

Table des matières

Introduction **3**

1 Introduction au modèle orienté colonnes **3**
1.1 Pourquoi ce modèle existe . . . 3
1.2 Une précision de vocabulaire . . . 3
1.3 Place dans la famille NoSQL . . . 4

2 Principe de fonctionnement : lignes, colonnes, familles de colonnes **4**
2.1 Le modèle de données de Bigtable . . . 4
2.2 Le vocabulaire dans Apache Cassandra . . . 4
2.3 Pourquoi « wide » ? . . . 5
2.4 Le cas d'Apache HBase . . . 5

3 Comparaison : stockage colonnaire, Column-Family et relationnel **5**

4 Architecture distribuée : partitionnement, réplication, tolérance aux pannes **6**
4.1 Partitionnement . . . 6
4.2 Réplication . . . 6
4.3 Cohérence ajustable . . . 7
4.4 Tolérance aux pannes . . . 7
4.5 Chemin d'écriture . . . 8
4.6 Chemin de lecture et suppressions . . . 8
4.7 Comparaison avec HBase . . . 8

5 Scalabilité et performances **9**
5.1 Une montée en charge par ajout de machines . . . 9
5.2 Opérations peu coûteuses et opérations coûteuses . . . 9
5.3 Les compromis . . . 9

6 Principaux systèmes **9**

7 Cas d'utilisation **10**

8 Étude de cas pratique : base Cassandra pour des capteurs IoT **11**
8.1 Énoncé et objectif . . . 11
8.2 Partir des requêtes . . . 11
8.3 Une table par requête . . . 11
8.4 Environnement . . . 12
8.5 Création du schéma . . . 12
8.6 Insertion des données . . . 14
8.7 Requêtes de lecture . . . 15
8.8 Analyse des données . . . 16
8.9 Limites : une requête non prévue . . . 17
8.10 Réplication et partitionnement . . . 18
8.11 Bilan de l'étude de cas . . . 20

9 Performances sur un historique de dix ans : Cassandra face au relationnel 20
9.1 Objectif . . . 20
9.2 Protocole . . . 20
9.3 Le modèle relationnel de comparaison . . . 21
9.4 Les requêtes comparées . . . 21
9.5 Volume stocké et chargement . . . 22
9.6 Temps de lecture selon le volume . . . 23
9.7 Aller chercher une donnée ancienne . . . 24
9.8 Requête non prévue sur tout l'historique . . . 27
9.9 Écritures unitaires . . . 28
9.10 Synthèse . . . 28

10 Étude comparative : modèle relationnel et modèle Wide-Column 29
10.1 Comparaison générale . . . 29
10.2 Études de cas comparatives . . . 29

11 Passer du modèle relationnel au modèle orienté colonnes 30
11.1 Est-ce possible ? . . . 30
11.2 La démarche . . . 31
11.3 Règles de correspondance . . . 31
11.4 Application à notre schéma . . . 31
11.5 Ce que l'on gagne et ce que l'on perd . . . 32

12 Questions et éléments de réponse 32

Conclusion 34

Introduction

Pendant longtemps, la réponse habituelle au stockage de données a été la base relationnelle : des tables, des jointures, des transactions, sur un serveur unique aussi puissant que possible. Ce modèle, formalisé par Codd en 1970 [11], reste adapté à un grand nombre d'applications. Il rencontre cependant une limite matérielle, car un serveur ne peut pas grandir indéfiniment. Lorsque des millions de capteurs ou d'utilisateurs écrivent en continu, il devient plus réaliste de répartir les données sur de nombreuses machines ordinaires, et de continuer à fonctionner quand l'une d'elles tombe en panne.

Deux publications ont orienté ce mouvement. En 2006, Google a décrit Bigtable [1], un système qui organise les données en lignes triées et en familles de colonnes, réparties sur un grand nombre de serveurs. En 2007, Amazon a décrit Dynamo [2], un magasin clé-valeur sans serveur maître, conçu pour rester disponible malgré les pannes. Facebook a combiné les deux approches dans Cassandra [3], publié en open source en 2008 puis repris par la fondation Apache : le modèle de données vient de Bigtable, l'architecture distribuée de Dynamo.

Ce document cherche à répondre à la question suivante : *comment les bases orientées colonnes permettent-elles de gérer efficacement des volumes massifs de données distribuées, et quels compromis impliquent-elles ?* Nous suivons pour cela les huit points du thème. Les sections 1 à 3 présentent le modèle et le situent par rapport au relationnel et au stockage colonnaire. Les sections 4 et 5 décrivent l'architecture distribuée et ses performances, les sections 6 et 7 les systèmes existants et leurs usages, et la section 8 met l'ensemble en pratique avec Cassandra. La section 9 mesure les performances du modèle sur un historique allant jusqu'à dix ans et les compare à celles d'une base relationnelle. La section 10 compare les deux modèles sur plusieurs études de cas, la section 11 explique comment passer de l'un à l'autre, et la section 12 propose des questions sur le thème avec leurs éléments de réponse.

Pour illustrer les notions, nous reprenons partout le même exemple : un réseau de capteurs de température et d'humidité, qui envoie une mesure toutes les dix secondes. Avec ces bases, on part des requêtes que l'application devra exécuter pour construire les tables, alors qu'en relationnel on modélise d'abord les données.

1 Introduction au modèle orienté colonnes

1.1 Pourquoi ce modèle existe

Les bases relationnelles offrent les jointures, les contraintes d'intégrité et les transactions ACID. Ces garanties ont un coût lorsque les données sont réparties : une jointure entre deux tables situées sur des machines différentes exige des échanges réseau, et une transaction qui touche plusieurs nœuds demande une coordination lente. Les bases orientées colonnes, comme d'autres bases NoSQL, renoncent à une partie de ces garanties. En échange, elles permettent d'augmenter la capacité en ajoutant des machines plutôt qu'en remplaçant celle qui existe (*scale out*), de rester disponibles malgré des pannes et d'absorber un débit d'écriture élevé.

1.2 Une précision de vocabulaire

Le mot « colonne » recouvre deux familles de systèmes. La première est le *stockage colonnaire* (*column store*), dans lequel les valeurs d'une même colonne sont écrites côte à côte sur le disque. Il vise l'analyse (OLAP), qui lit peu de colonnes sur beaucoup de lignes ; ClickHouse, Amazon Redshift et le format Parquet en sont des exemples. La seconde est le modèle *Wide-Column*, aussi appelé *Column-Family*,

dans lequel les données sont des lignes identifiées par une clé, dont les colonnes sont regroupées en familles et peuvent différer d'une ligne à l'autre. Il vise la distribution et l'écriture massive ; Bigtable, Cassandra, HBase et ScyllaDB en relèvent. Notre thème est la seconde famille. La première n'intervient que dans la comparaison de la section 3.

Dans la suite, « Wide-Column » désigne ce modèle, tandis que « famille de colonnes » renvoie à un objet précis de Bigtable et de HBase (voir la section 2.2).

1.3 Place dans la famille NoSQL

NoSQL (*Not only SQL*) regroupe quatre grands modèles : clé-valeur, document, graphe et colonnes. Le modèle Wide-Column se situe entre le clé-valeur, dont il garde l'accès par clé et la distribution, et le relationnel, dont il garde l'idée de lignes et de colonnes typées, mais sans les jointures.

2 Principe de fonctionnement : lignes, colonnes, familles de colonnes

2.1 Le modèle de données de Bigtable

Le modèle décrit par Google [1] est une table éparse, distribuée et triée, qui associe une valeur à trois coordonnées :

$$(\textit{clé de ligne}, \textit{famille :colonne}, \textit{horodatage}) \longmapsto \textit{valeur}.$$

Les lignes sont triées par leur clé, ce qui rend efficaces les lectures par plage. Les colonnes sont regroupées en familles, déclarées à l'avance, alors que les colonnes elles-mêmes peuvent être ajoutées librement à l'intérieur d'une famille. Une cellule peut aussi conserver plusieurs versions, distinguées par leur horodatage. La table est dite éparse parce qu'une ligne qui n'a pas de valeur pour une colonne ne consomme aucune place pour elle : deux lignes d'une même table peuvent donc avoir des colonnes différentes.

2.2 Le vocabulaire dans Apache Cassandra

Cassandra reprend cette organisation et l'expose à travers un langage proche de SQL, le CQL (*Cassandra Query Language*). Le tableau ci-dessous donne le vocabulaire, du plus englobant au plus fin.

Élément	Rôle
Cluster	Ensemble des machines (nœuds) qui forment la base.
Keyspace	Espace de noms qui regroupe des tables et fixe la stratégie de réplication (équivalent d'une base de données).
Table	Collection de partitions décrite par un schéma (colonnes et types).
Partition	Groupe de lignes qui partagent la même clé de partition. C'est l'unité de répartition entre les machines.
Ligne	Entrée identifiée dans sa partition par sa clé de clustering.
Cellule	Valeur d'une colonne, avec un horodatage d'écriture et une durée de vie facultative (*TTL*).

La clé primaire d'une table Cassandra comporte deux parties. La *clé de partition* détermine sur quelles machines la donnée est stockée. Les *clés de clustering* déterminent l'ordre des lignes à l'intérieur d'une partition. La modélisation repose en grande partie sur cette distinction, comme le montre l'étude de cas.

Le vocabulaire varie d'un système à l'autre. Dans Bigtable et HBase, la famille de colonnes est un objet du schéma, déclaré à la création de la table. Cassandra n'a pas de famille de colonnes en ce sens : avant CQL 3, le terme *column family* y désignait ce qu'on appelle aujourd'hui une table. C'est la partition qui y joue le rôle de ligne extensible.

2.3 Pourquoi « wide » ?

Une partition peut contenir un très grand nombre de lignes, donc de cellules, d'où l'adjectif *wide*. Dans notre exemple, la partition d'un capteur pour une journée regroupe toutes ses mesures, de la plus récente à la plus ancienne (figure 1).

Clé de partition `(S001, 2026-10-05)`				
	`ts=08:00:20`	`temp=27.9`	`hum=60.5`	`bat=88`
	`ts=08:00:10`	`temp=27.6`	`hum=60.8`	`bat=88`
	`ts=08:00:00`	`temp=27.4`	`hum=61.0`	`bat=88`

... des milliers de lignes, triées par la clé de clustering `ts` *(ordre décroissant)*

FIGURE 1 – Une partition de l'exemple : toutes les mesures d'un capteur pour un jour, stockées ensemble et triées.

2.4 Le cas d'Apache HBase

HBase, inspiré directement de Bigtable, en garde le modèle : une table, des lignes triées par clé, des familles de colonnes, des versions horodatées. Chaque famille y est stockée dans ses propres fichiers, ce qui conduit à en déclarer peu et à regrouper dans une même famille les colonnes lues ensemble.

3 Comparaison : stockage colonnaire, Column-Family et relationnel

La figure 2 montre la même information rangée de trois manières. Le tableau qui suit en tire les conséquences.

1. Relationnel (stockage par ligne) : ***une ligne = un bloc***

1 | Ali | Cotonou | 1500	2 | Bio | Abomey | 2300	3 | Zola | Ouidah | 850

2. Stockage colonnaire (analytique) : ***une colonne = un bloc***

1 | 2 | 3	Ali | Bio | Zola	Cotonou | Abomey | Ouidah	1500 | 2300 | 850

3. Wide-Column / Column-Family (Cassandra, HBase) : ***une clé + des cellules triées***

clé `S001`	ts1 : temp=27.4	ts1 : hum=61	ts2 : temp=27.6	ts2 : hum=60

Chaque clé peut avoir ses propres colonnes : les lignes n'ont pas besoin d'être identiques.

FIGURE 2 – Une même donnée, trois organisations physiques.

Critère	Relationnel (ligne)	Stockage colonnaire	Wide-Column (Column-Family)
Unité de stockage	La ligne complète	La colonne complète	La partition (clé + cellules triées)
Usage visé	Transactions (OLTP)	Analyse (OLAP)	Écriture et lecture massives, distribuées
Schéma	Fixe, imposé	Fixe, imposé	Familles fixes, colonnes souples
Jointures	Oui, natives	Oui (SQL analytique)	Non, évitées par la modélisation
Transactions	ACID complètes	Limitées	Limitées (par partition, ou *lightweight*)
Distribution	Possible, mais difficile	Selon le système	Native, dès la conception
Modélisation guidée par	Les données (normalisation)	Les analyses à mener	Les requêtes prévues
Exemples	PostgreSQL, MySQL	ClickHouse, Redshift	Cassandra, HBase, ScyllaDB

Dans Cassandra, les cellules d'une même partition sont stockées ensemble, et non colonne par colonne sur l'ensemble du disque. L'appellation vient du modèle de données de Bigtable, pas de la disposition physique des octets. Le modèle abandonne par ailleurs les jointures de façon délibérée : cette fonction est échangée contre la distribution des données.

4 Architecture distribuée : partitionnement, réplication, tolérance aux pannes

Nous décrivons ici l'architecture de Cassandra [3, 7], puis nous la rapprochons de celle de HBase.

4.1 Partitionnement

Cassandra n'a pas de serveur maître : tous les nœuds ont le même rôle. Pour placer une donnée, le système applique une fonction de hachage (Murmur3) à la clé de partition. Le résultat, appelé *token*, est un entier compris entre -2^{63} et $2^{63}-1$. Ces valeurs sont disposées sur un anneau, et chaque nœud est responsable d'une plage de tokens ; le propriétaire d'une clé se retrouve donc par un calcul, sans annuaire central. Ce hachage cohérent a un avantage : lorsqu'on ajoute un nœud, seule une fraction des données change de propriétaire. Pour équilibrer la charge, chaque nœud reçoit en pratique plusieurs petites plages, les *vnodes*.

4.2 Réplication

Chaque partition est copiée sur plusieurs nœuds. Le nombre de copies est le facteur de réplication (RF), fixé au niveau du keyspace. Avec RF = 3, la donnée est stockée sur le nœud propriétaire de son token et sur les deux suivants de l'anneau, de préférence dans des baies ou des datacenters distincts (figure 3). Les copies sont équivalentes : aucune n'est « principale ». La stratégie `NetworkTopologyStrategy` permet de fixer le nombre de copies par datacenter.

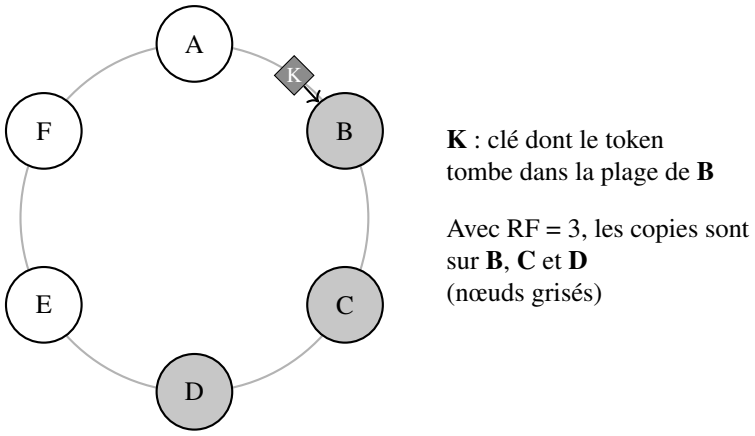


FIGURE 3 – Anneau de six nœuds et réplication avec RF = 3.

4.3 Cohérence ajustable

Puisqu'il existe plusieurs copies, il faut décider combien d'entre elles doivent répondre avant de confirmer une opération. Cassandra laisse ce choix à l'application, requête par requête, à travers le *niveau de cohérence*.

Niveau	Signification	Compromis
`ONE`	Une seule copie répond	Le plus rapide ; risque de lire une donnée un peu ancienne
`QUORUM`	La majorité des copies ($\lfloor RF/2 \rfloor + 1$)	Équilibre courant ; 2 copies si RF = 3
`LOCAL_QUORUM`	Majorité dans le datacenter local	Évite la latence entre sites distants
`ALL`	Toutes les copies	Cohérence maximale ; une seule panne bloque la requête

Une règle simple garantit de lire la dernière valeur écrite. Si W copies confirment l'écriture et que R copies sont interrogées en lecture, alors

$$R + W > RF \implies \text{la lecture voit toujours l'écriture la plus récente.}$$

Avec RF = 3, écrire puis lire en `QUORUM` donne $2+2=4>3$: la lecture est cohérente et le système supporte la panne d'un nœud.

Ce réglage illustre le théorème CAP [5] : lors d'une partition du réseau, un système distribué doit renoncer soit à la cohérence, soit à la disponibilité. Par défaut, Cassandra privilégie la disponibilité, et l'application se rapproche de la cohérence en relevant le niveau. Le modèle PACELC [6] ajoute que, même sans panne, il subsiste un compromis entre latence et cohérence.

4.4 Tolérance aux pannes

Cassandra n'a pas de point de défaillance unique. Quatre mécanismes se complètent :

— le *gossip* : chaque nœud échange régulièrement avec quelques voisins l'état de tous les autres, ce qui permet de détecter une panne ;

— le *hinted handoff* : si une copie est indisponible pendant une écriture, un autre nœud garde une note (*hint*) et la transmet au retour du nœud, dans une limite de temps (3 heures par défaut) ;

— le *read repair* : lors d'une lecture, si les copies interrogées diffèrent, la plus récente est propagée aux autres ;

— l'*anti-entropy repair* : une opération périodique (`nodetool repair`) compare les copies avec des arbres de Merkle et corrige les écarts restants.

4.5 Chemin d'écriture

Cassandra repose sur une structure appelée *LSM-tree* (*Log-Structured Merge tree*) [4], dont le principe est de ne jamais modifier un fichier existant mais d'écrire à la suite (figure 4).

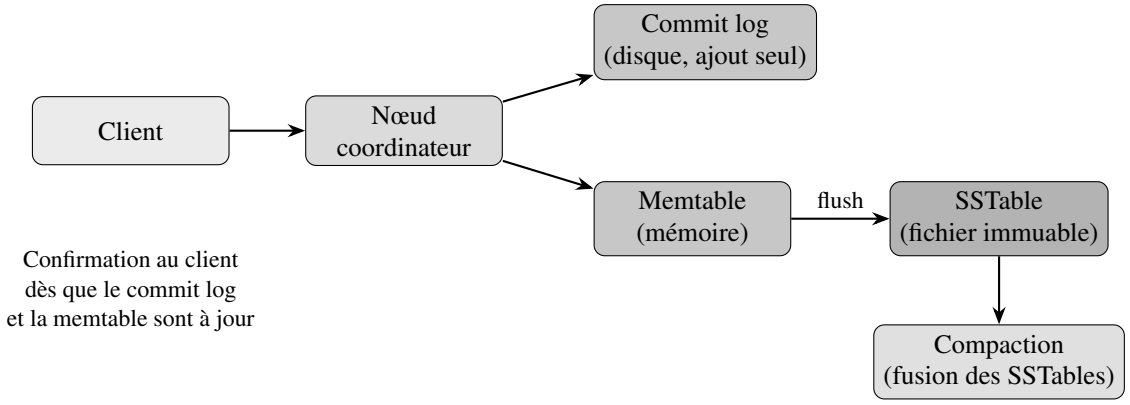


FIGURE 4 – Chemin d'écriture dans Cassandra.

1. L'écriture est ajoutée au *commit log*, journal écrit séquentiellement sur disque, qui garantit la durabilité en cas de panne.
2. Elle est aussi insérée dans la *memtable*, structure triée en mémoire.
3. Le client reçoit alors sa confirmation, sans qu'aucune lecture préalable ni modification sur place n'ait été nécessaire.
4. Quand la memtable est pleine, elle est écrite sur disque en un fichier trié et immuable, la *SSTable* (*flush*).
5. En arrière-plan, la *compaction* fusionne les SSTables pour éliminer les doublons et les données périmées.

4.6 Chemin de lecture et suppressions

Pour lire une partition, Cassandra consulte la memtable et éventuellement plusieurs SSTables, puis fusionne les résultats en gardant, pour chaque cellule, l'horodatage le plus récent (*last write wins*). Un filtre de Bloom indique qu'une SSTable ne contient certainement pas la partition demandée, ce qui évite des lectures disque inutiles.

Comme les fichiers sont immuables, une suppression n'efface rien sur le moment. Elle écrit un marqueur, la *tombstone*, qui masque la donnée jusqu'à la compaction, après un délai de sécurité (`gc_grace_seconds`, 10 jours par défaut).

4.7 Comparaison avec HBase

HBase adopte une organisation différente [8]. Un *HMaster* et des *RegionServers*, coordonnés par ZooKeeper, se partagent des régions, c'est-à-dire des plages contiguës de clés triées, et les fichiers sont

stockés sur le système distribué HDFS. La cohérence est forte au niveau de la ligne, puisqu'un seul RegionServer sert une région à un instant donné. L'écriture reprend le même principe LSM (WAL, MemStore, HFile). Le piège classique est proche de celui de Cassandra : des clés qui croissent de façon monotone concentrent toutes les écritures sur une seule région. Le tableau de la section 6 résume les différences.

5 Scalabilité et performances

5.1 Une montée en charge par ajout de machines

Les données étant réparties par hachage et tous les nœuds équivalents, augmenter la capacité revient à ajouter un nœud : il prend en charge une part de l'anneau, et le débit total croît à peu près proportionnellement au nombre de nœuds. Cette croissance quasi linéaire est une propriété visée par la conception. Ses valeurs précises dépendent du matériel, du modèle de données et de la charge, et doivent être mesurées plutôt que supposées.

5.2 Opérations peu coûteuses et opérations coûteuses

Cette architecture rend peu coûteuses les écritures en grand nombre, la lecture d'une partition ou d'une plage à l'intérieur d'une partition, la disponibilité pendant une panne et l'expiration automatique des données par TTL. En revanche, elle rend coûteuses ou impossibles les lectures sans la clé de partition, les jointures et agrégations entre tables, les transactions ACID sur plusieurs lignes et les suppressions massives, qui produisent beaucoup de tombstones. Une partition qui grossit sans limite, ou qui concentre à elle seule une grande part de la charge, dégrade aussi les performances.

5.3 Les compromis

Le tableau ci-dessous résume ce que l'on obtient et ce que l'on paie.

Avantage	Contrepartie
Scalabilité horizontale et haute disponibilité	Cohérence forte non automatique : il faut choisir les niveaux de cohérence
Écritures très rapides (LSM-tree)	Lectures plus coûteuses (plusieurs fichiers à fusionner) et compaction en arrière-plan
Aucune jointure à exécuter à la lecture	Duplication des données : une table par type de requête
Schéma souple (colonnes variables)	Modélisation plus exigeante : une requête non prévue est coûteuse ou refusée

La complexité qui se trouvait au moment de la lecture (jointures, agrégations) est déplacée vers la conception des tables et vers l'écriture. Une mauvaise conception se corrige donc rarement par une requête plus astucieuse ; il faut souvent recréer la table.

6 Principaux systèmes

Apache Cassandra est né chez Facebook en 2008 et est devenu un projet Apache. Écrit en Java, il repose sur une architecture pair-à-pair, le langage CQL, une cohérence ajustable et une réplication multi-datacenter. C'est l'une des implémentations les plus utilisées du modèle Wide-Column.

Apache HBase est l'implémentation open source du modèle Bigtable dans l'écosystème Hadoop. Il s'appuie sur HDFS, offre une cohérence forte par ligne et s'intègre bien à Spark et à MapReduce, au prix d'un déploiement plus lourd (HDFS, ZooKeeper).

ScyllaDB est une réécriture de Cassandra en C++ [9], compatible avec CQL. Son architecture attribue à chaque cœur de processeur sa propre portion de données (*shard per core*), ce qui évite les pauses du ramasse-miettes de la JVM. Le fabricant annonce une latence plus faible avec moins de machines ; ce gain dépend de la charge et doit être vérifié par un test. La licence en vigueur est à consulter avant tout usage professionnel.

Google Cloud Bigtable, enfin, est le service géré issu du système d'origine, accessible avec l'API de HBase.

	Cassandra	HBase	ScyllaDB
Langage	Java	Java	C++
Architecture	Pair-à-pair	Maître + régions, HDFS	Pair-à-pair, un fragment par cœur
Interface	CQL	API HBase, shell	CQL (compatible Cassandra)
Cohérence	Ajustable	Forte par ligne	Ajustable
Point fort	Disponibilité, multi-datacenter	Intégration Hadoop et Spark	Performance par machine
Limite	Réglage de la JVM, compaction	Déploiement complexe	Écosystème plus restreint

7 Cas d'utilisation

Les bons usages ont en commun de nombreuses écritures, des requêtes connues à l'avance et un besoin de distribution ou de disponibilité.

Domaine	Raison de l'adéquation	Partition type
IoT	Flux continu de mesures, lecture surtout par capteur et par période	(capteur, jour)
Journaux (logs)	Volume important, écriture en ajout, consultation par service et par date	(service, heure)
Séries temporelles	Données horodatées, tri naturel par temps, expiration automatique	(métrique, jour)
Big Data	Stockage réparti sur de nombreuses machines, traitement par lots avec Spark	Clé de ligne bien répartie
Télécommunications	Enregistrements d'appels et de consommation, gros volumes, disponibilité permanente	(abonné, mois)

À l'inverse, ces bases conviennent mal aux applications qui reposent sur des transactions ACID entre plusieurs tables, comme la comptabilité ou les virements. Elles conviennent mal aussi à l'analyse libre, lorsqu'on ne sait pas d'avance quelles questions seront posées : un entrepôt colonnaire est alors préférable. Enfin, pour de petits volumes, un serveur relationnel suffit et reste plus simple à exploiter.

8 Étude de cas pratique : base Cassandra pour des capteurs IoT

8.1 Énoncé et objectif

Une entreprise déploie des capteurs de température et d'humidité dans plusieurs zones d'une ville. Chaque capteur envoie une mesure toutes les 10 secondes. Nous concevons la base Cassandra qui stocke ces mesures et permet de les exploiter, c'est-à-dire de les interroger et d'en tirer des indicateurs. L'objectif est de montrer que la modélisation dépend des requêtes prévues.

8.2 Partir des requêtes

Avant de créer une table, nous listons ce que l'application devra faire. Cette démarche, qui place les requêtes avant les tables, correspond aux méthodologies de modélisation pour Cassandra [10]. Les quatre premières requêtes servent à consulter les mesures, les deux dernières à les analyser.

N°	Requête métier
Q1	Afficher les dernières mesures d'un capteur pour un jour donné.
Q2	Afficher les mesures d'un capteur entre deux heures d'une journée.
Q3	Connaître la dernière mesure de chaque capteur (tableau de bord).
Q4	Lister les alertes (température supérieure à 35 °C) d'une zone pour un jour donné.
Q5	Calculer, pour un capteur et un jour donnés, le nombre de mesures et les températures minimale, maximale et moyenne.
Q6	Suivre l'évolution de ces indicateurs, jour par jour, sur une période.

8.3 Une table par requête

À chaque type de requête correspond une table dont la clé primaire est choisie pour la servir directement. Q5 peut s'appuyer sur la table des mesures, car elle ne porte que sur une partition ; Q6 demande une table propre, pour les raisons exposées en section 8.8.

Table	Clé primaire	Requêtes servies
`readings_by_sensor_day`	`((sensor_id, day), ts)`	Q1, Q2, Q5
`last_reading_by_sensor`	`sensor_id`	Q3
`alerts_by_zone_day`	`((zone, day), ts, sensor_id)`	Q4
`daily_stats_by_sensor`	`(sensor_id, day)`	Q6

Le jour figure dans la clé de partition de la première table pour une raison de taille. À une mesure toutes les 10 secondes, un capteur produit $6 \times 60 \times 24 = 8\,640$ mesures par jour, soit environ $8\,640 \times 365 \approx 3{,}15$ millions par an. Sans le jour, toutes ces mesures seraient dans une seule partition qui grossirait sans fin, ralentirait les lectures et la compaction, et ne pourrait pas être répartie sur plusieurs machines. Avec le jour, chaque partition reste bornée à environ 8 640 lignes. Cette technique s'appelle le *bucketing*. Une règle empirique souvent citée est de garder les partitions sous 100 Mo et sous une centaine de milliers de lignes.

8.4 Environnement

Nous utilisons Docker, pour tester Cassandra sans l'installer.

Code 1 – Démarrage d'un nœud Cassandra

```
docker run -d --name cass1 -p 9042:9042 cassandra:5.0
# Attendre 1 a 2 minutes, puis verifier :
docker exec cass1 nodetool status
docker exec -it cass1 cqlsh
```

Pour illustrer la réplication, nous lançons un cluster de trois nœuds. Chaque nœud demande de la mémoire, d'où la limitation du tas Java.

Code 2 – Cluster de trois nœuds

```
docker network create cass-net
docker run -d --name cass1 --network cass-net \
  -e MAX_HEAP_SIZE=512M -e HEAP_NEWSIZE=128M cassandra:5.0
# attendre que cass1 soit "UN" (Up/Normal) dans nodetool status
docker run -d --name cass2 --network cass-net -e CASSANDRA_SEEDS=cass1 \
  -e MAX_HEAP_SIZE=512M -e HEAP_NEWSIZE=128M cassandra:5.0
# attendre que cass2 soit "UN", puis :
docker run -d --name cass3 --network cass-net -e CASSANDRA_SEEDS=cass1 \
  -e MAX_HEAP_SIZE=512M -e HEAP_NEWSIZE=128M cassandra:5.0
```

Les trois conteneurs s'exécutent sur la même machine, un poste Linux. Les essais montrent donc le comportement logique du cluster (répartition, réplication, panne d'un nœud), mais les temps de réponse et les pannes réseau ne sont pas représentatifs d'un déploiement réparti sur plusieurs sites. La commande `nodetool status` confirme que les trois nœuds sont à l'état `UN` (figure 5). Les pourcentages de la colonne *Owns* ne se rapportent pas au keyspace `iot`, que la commande ne précise pas.

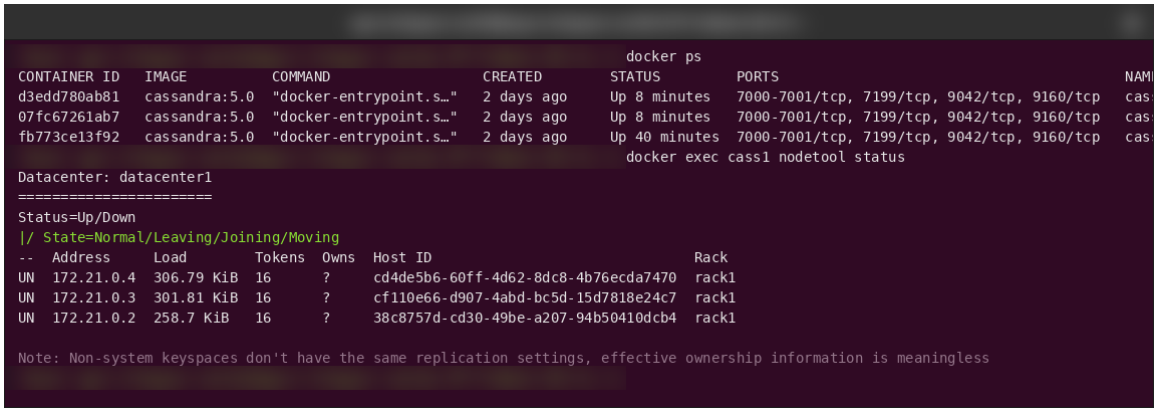


FIGURE 5 – Résultat de `nodetool status` : les 3 nœuds sont à l'état UN (Up/Normal).

8.5 Création du schéma

Code 3 – Keyspace et tables (CQL)

```
// === Cluster de 3 noeuds ===
CREATE KEYSPACE IF NOT EXISTS iot
WITH replication = {
  'class': 'NetworkTopologyStrategy',
  'datacenter1': 3
```

```
};

// Sinon, pour un seul noeud (test uniquement) :
// CREATE KEYSPACE IF NOT EXISTS iot
// WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

USE iot;

// === Q1, Q2 et Q5 : mesures d'un capteur, par jour, de la plus recente a la plus
    ancienne ===
CREATE TABLE readings_by_sensor_day (
  sensor_id text,
  day date,
  ts timestamp,
  temperature double,
  humidity double,
  battery_pct int,
  PRIMARY KEY ((sensor_id, day), ts)
) WITH CLUSTERING ORDER BY (ts DESC)
  AND default_time_to_live = 7776000   // expiration apres 90 jours
  AND compaction = {
    'class': 'TimeWindowCompactionStrategy',
    'compaction_window_unit': 'DAYS',
    'compaction_window_size': 1
  };

// === Q3 : derniere mesure par capteur (une ligne par capteur) ===
CREATE TABLE last_reading_by_sensor (
  sensor_id text PRIMARY KEY,
  ts timestamp,
  temperature double,
  humidity double
);

// === Q4 : alertes par zone et par jour ===
CREATE TABLE alerts_by_zone_day (
  zone text,
  day date,
  ts timestamp,
  sensor_id text,
  temperature double,
  PRIMARY KEY ((zone, day), ts, sensor_id)
) WITH CLUSTERING ORDER BY (ts DESC, sensor_id ASC);

// === Q6 : indicateurs journaliers par capteur (une ligne par jour) ===
CREATE TABLE daily_stats_by_sensor (
  sensor_id text,
  day date,
  nb_mesures int,
  temp_min double,
  temp_max double,
  temp_moy double,
  PRIMARY KEY (sensor_id, day)
) WITH CLUSTERING ORDER BY (day DESC);
```

L'ordre de clustering décroissant (`ts DESC`) range les mesures de la plus récente à la plus ancienne, de sorte que Q1 lit directement les premières lignes sans tri supplémentaire. La stratégie de compaction par fenêtres de temps (*TimeWindowCompactionStrategy*) regroupe les données par période, ce qui convient aux séries temporelles avec expiration, puisqu'on peut supprimer des fichiers entiers sans les réécrire. La colonne `ts` de `last_reading_by_sensor` date la dernière mesure connue d'un capteur ; sans elle, on ne saurait pas si la valeur est fraîche ou vieille de plusieurs heures. Dans `daily_stats_by_sensor`, la partition est le capteur et les jours sont triés par clustering : une période se lit donc comme une plage à l'intérieur d'une seule partition.

Le keyspace utilise `NetworkTopologyStrategy` avec un facteur de réplication de 3, en cohérence avec le cluster de trois nœuds déployé en section 8.4 et testé en section 8.10.

8.6 Insertion des données

Une même mesure alimente plusieurs tables : la duplication est voulue. Chaque mesure est donc écrite dans un `BATCH` logué, qui regroupe les écritures liées. Une fois le lot accepté, Cassandra garantit que toutes ses écritures finiront par être appliquées (atomicité). Ce lot touche ici deux ou trois partitions différentes, ce qui reste raisonnable.

Code 4 – Insertion de mesures

```
// S001 : trois mesures normales
BEGIN BATCH
  INSERT INTO readings_by_sensor_day (sensor_id, day, ts, temperature, humidity,
    battery_pct)
    VALUES ('S001', '2026-10-05', '2026-10-05 08:00:00+0000', 27.4, 61.0, 88);
  INSERT INTO last_reading_by_sensor (sensor_id, ts, temperature, humidity)
    VALUES ('S001', '2026-10-05 08:00:00+0000', 27.4, 61.0);
APPLY BATCH;

BEGIN BATCH
  INSERT INTO readings_by_sensor_day (sensor_id, day, ts, temperature, humidity,
    battery_pct)
    VALUES ('S001', '2026-10-05', '2026-10-05 08:00:10+0000', 27.6, 60.8, 88);
  INSERT INTO last_reading_by_sensor (sensor_id, ts, temperature, humidity)
    VALUES ('S001', '2026-10-05 08:00:10+0000', 27.6, 60.8);
APPLY BATCH;

BEGIN BATCH
  INSERT INTO readings_by_sensor_day (sensor_id, day, ts, temperature, humidity,
    battery_pct)
    VALUES ('S001', '2026-10-05', '2026-10-05 08:00:20+0000', 27.9, 60.5, 88);
  INSERT INTO last_reading_by_sensor (sensor_id, ts, temperature, humidity)
    VALUES ('S001', '2026-10-05 08:00:20+0000', 27.9, 60.5);
APPLY BATCH;

// S002 : une mesure normale, puis une mesure au-dessus du seuil de 35 degres
BEGIN BATCH
  INSERT INTO readings_by_sensor_day (sensor_id, day, ts, temperature, humidity,
    battery_pct)
    VALUES ('S002', '2026-10-05', '2026-10-05 08:00:00+0000', 30.1, 55.2, 91);
  INSERT INTO last_reading_by_sensor (sensor_id, ts, temperature, humidity)
    VALUES ('S002', '2026-10-05 08:00:00+0000', 30.1, 55.2);
APPLY BATCH;
```

```
// La mesure anormale alimente aussi la table des alertes
BEGIN BATCH
  INSERT INTO readings_by_sensor_day (sensor_id, day, ts, temperature, humidity,
    battery_pct)
    VALUES ('S002', '2026-10-05', '2026-10-05 08:00:10+0000', 36.4, 52.0, 91);
  INSERT INTO last_reading_by_sensor (sensor_id, ts, temperature, humidity)
    VALUES ('S002', '2026-10-05 08:00:10+0000', 36.4, 52.0);
  INSERT INTO alerts_by_zone_day (zone, day, ts, sensor_id, temperature)
    VALUES ('Zone-Nord', '2026-10-05', '2026-10-05 08:00:10+0000', 'S002', 36.4);
APPLY BATCH;
```

Ce coût de coordination (le coordinateur journalise le lot avant de l'exécuter) explique pourquoi le `BATCH` sert à garder cohérentes quelques écritures liées, et non à accélérer des insertions en masse. Pour de gros volumes, l'application enverrait les mesures une par une, ou par petits lots ne touchant qu'une seule partition. Garder les tables synchronisées reste un travail de l'application : si l'une des écritures était oubliée, Q3 renverrait une valeur périmée alors que Q1 afficherait des mesures plus récentes. Le `BATCH` réduit ce risque sans le supprimer, et c'est une conséquence directe de la duplication.

8.7 Requêtes de lecture

Code 5 – Requêtes Q1 à Q4

```
// Q1 : dernieres mesures d'un capteur pour un jour
SELECT ts, temperature, humidity FROM readings_by_sensor_day
  WHERE sensor_id = 'S001' AND day = '2026-10-05' LIMIT 10;

// Q2 : mesures entre deux heures
SELECT ts, temperature FROM readings_by_sensor_day
  WHERE sensor_id = 'S001' AND day = '2026-10-05'
    AND ts >= '2026-10-05 08:00:00+0000' AND ts < '2026-10-05 12:00:00+0000';

// Q3 : derniere mesure d'un capteur
SELECT * FROM last_reading_by_sensor WHERE sensor_id = 'S001';

// Q4 : alertes d'une zone pour un jour
SELECT * FROM alerts_by_zone_day WHERE zone = 'Zone-Nord' AND day = '2026-10-05';
```



FIGURE 6 – Résultats de Q1 et Q2 dans `cqlsh` : les mesures sont servies par ordre décroissant (Q1) et la plage temporelle est respectée (Q2).



FIGURE 7 – Résultats de Q3 et Q4 dans `cqlsh` : la dernière mesure de S001 (08 :00 :20) est la même que la première ligne de Q1, et l'alerte de la zone concerne S002.

Ces requêtes sont rapides parce qu'elles fournissent la clé de partition complète : Cassandra sait exactement quels nœuds interroger et lit une plage contiguë de lignes déjà triées.

8.8 Analyse des données

Le thème demande de stocker, mais aussi d'analyser les données de capteurs. Cassandra n'est pas un moteur d'analyse. Elle sait toutefois calculer des agrégats (`COUNT`, `MIN`, `MAX`, `SUM`, `AVG`) sur les lignes qu'elle lit, et ce calcul est peu coûteux tant qu'il reste dans une seule partition. C'est le cas de Q5 : la partition d'un capteur pour un jour contient au plus 8 640 lignes.

Pour Q6, répéter Q5 sur trente partitions-jour à chaque affichage multiplierait les lectures. Nous précalculons donc les indicateurs de chaque journée, par un traitement de l'application en fin de journée, dans la table `daily_stats_by_sensor`. Une période se lit alors dans une seule partition, qui ne compte qu'une ligne par jour, soit environ 365 lignes par an et par capteur.

Code 6 – Indicateurs journaliers, Q5 et Q6

```
// Indicateurs precalcules (jours 03 et 04 : valeurs d'exemple ; jour 05 : calcule
    sur les 3 mesures de S001)
```

```
INSERT INTO daily_stats_by_sensor (sensor_id, day, nb_mesures, temp_min, temp_max,
    temp_moy)
  VALUES ('S001', '2026-10-03', 8640, 24.9, 31.2, 27.8);
INSERT INTO daily_stats_by_sensor (sensor_id, day, nb_mesures, temp_min, temp_max,
    temp_moy)
  VALUES ('S001', '2026-10-04', 8640, 25.1, 32.0, 28.1);
INSERT INTO daily_stats_by_sensor (sensor_id, day, nb_mesures, temp_min, temp_max,
    temp_moy)
  VALUES ('S001', '2026-10-05', 3, 27.4, 27.9, 27.63);

// Q5 : indicateurs d'un capteur pour un jour (agregats sur une seule partition)
SELECT COUNT(*) AS nb_mesures, MIN(temperature) AS t_min,
       MAX(temperature) AS t_max, AVG(temperature) AS t_moy
  FROM readings_by_sensor_day
  WHERE sensor_id = 'S001' AND day = '2026-10-05';

// Q6 : evolution journaliere sur une periode (plage sur la cle de clustering)
SELECT day, nb_mesures, temp_min, temp_max, temp_moy FROM daily_stats_by_sensor
  WHERE sensor_id = 'S001' AND day >= '2026-10-03' AND day <= '2026-10-05';
```

```
cqlsh:iot> SELECT COUNT(*) AS nb_mesures, MIN(temperature) AS t_min,
       MAX(temperature) AS t_max, AVG(temperature) AS t_moy
  FROM readings_by_sensor_day
  WHERE sensor_id = 'S001' AND day = '2026-10-05';

 nb_mesures | t_min | t_max | t_moy
------------+-------+-------+----------
          3 |  27.4 |  27.9 | 27.63333

(1 rows)
cqlsh:iot> SELECT day, nb_mesures, temp_min, temp_max, temp_moy FROM daily_stats_by_sensor
  WHERE sensor_id = 'S001' AND day >= '2026-10-03' AND day <= '2026-10-05';

 day        | nb_mesures | temp_min | temp_max | temp_moy
------------+------------+----------+----------+----------
 2026-10-05 |          3 |     27.4 |     27.9 |    27.63
 2026-10-04 |       8640 |     25.1 |       32 |     28.1
 2026-10-03 |       8640 |     24.9 |     31.2 |     27.8

(3 rows)
cqlsh:iot>
```

FIGURE 8 – Résultats de Q5 (agrégats sur une partition) et de Q6 (indicateurs journaliers) dans `cqlsh`.

Une question comme « quelle est la température moyenne de tous les capteurs ? » reste hors de portée d'une requête ordinaire : elle obligerait à parcourir toutes les partitions de tous les nœuds. Ce type d'analyse relève d'un traitement distribué, par exemple Spark avec le connecteur Cassandra, ou d'un export vers un entrepôt colonnaire comme ClickHouse. On retrouve ici la distinction faite en section 3 : le Wide-Column sert à stocker et à consulter à grande échelle, le stockage colonnaire à analyser librement.

8.9 Limites : une requête non prévue

Interrogeons maintenant la base d'une manière que le modèle n'a pas prévue.

Code 7 – Requête non prévue par le modèle

```
// "Quelles mesures depassent 35 degres, tous capteurs confondus ?"
SELECT * FROM readings_by_sensor_day WHERE temperature > 35;
```

Cassandra refuse cette requête avec une erreur du type suivant.



FIGURE 9 – Refus de la requête « Quelles mesures dépassent 35 degrés ? » : Cassandra exige l'utilisation de `ALLOW FILTERING` pour éviter des performances imprévisibles.

Ce refus est voulu. Sans clé de partition, Cassandra devrait parcourir toutes les partitions de tous les nœuds. Avec `ALLOW FILTERING`, la requête s'exécute et renvoie la mesure de S002 à 36,4 °C, mais son coût croît avec la taille de la base : négligeable sur quelques lignes de test, très lourd sur des milliards de lignes.

Obtenir les mesures d'un capteur sur plusieurs jours demande d'interroger chaque partition-jour, par exemple avec `day IN (...)` ou par une requête par jour côté application ; le découpage par jour est donc un choix qu'il faut assumer. Quant à la question sur les températures élevées, elle trouve sa réponse dans une autre table conçue pour elle, ici `alerts_by_zone_day`. Si une nouvelle question apparaît, on crée une nouvelle table, on l'alimente dès maintenant et l'on migre l'historique.

8.10 Réplication et partitionnement

Le keyspace `iot` est configuré avec RF = 3 sur un cluster de trois nœuds : chaque nœud détient une copie de chaque partition. Cette configuration met en évidence la réplication et la tolérance aux pannes, mais pas le partitionnement, puisqu'aucun nœud n'a de part qui lui soit propre. La commande `nodetool status iot` l'indique : chaque nœud possède 100 % des données du keyspace.

Pour observer le partitionnement, nous créons un second keyspace de test, `iot_part`, avec RF = 1. Chaque partition n'existe alors que sur un seul nœud, désigné par le hachage de sa clé. La commande `nodetool getendpoints` prend en argument la clé de partition (et non son token) et indique le nœud responsable. Les clés de la table de test sont de simples identifiants de capteurs.

Code 8 – Keyspace de test sans réplication (dans cqlsh)

```
CREATE KEYSPACE IF NOT EXISTS iot_part
  WITH replication = {'class': 'NetworkTopologyStrategy', 'datacenter1': 1};

CREATE TABLE iot_part.readings_demo (
  sensor_id text,
  ts timestamp,
  temperature double,
  PRIMARY KEY (sensor_id, ts)
);
```

Code 9 – Observation de la répartition (dans le terminal)

```
# Part de donnees de chaque noeud : keyspace replique (RF = 3) puis keyspace de
    test (RF = 1)
docker exec cass1 nodetool status iot
docker exec cass1 nodetool status iot_part

# Noeud responsable de chaque capteur dans le keyspace de test
for k in S001 S002 S003 S004 S005 S006; do
  echo -n "$k : "
```

```
  docker exec cass1 nodetool getendpoints iot_part readings_demo $k
done
```

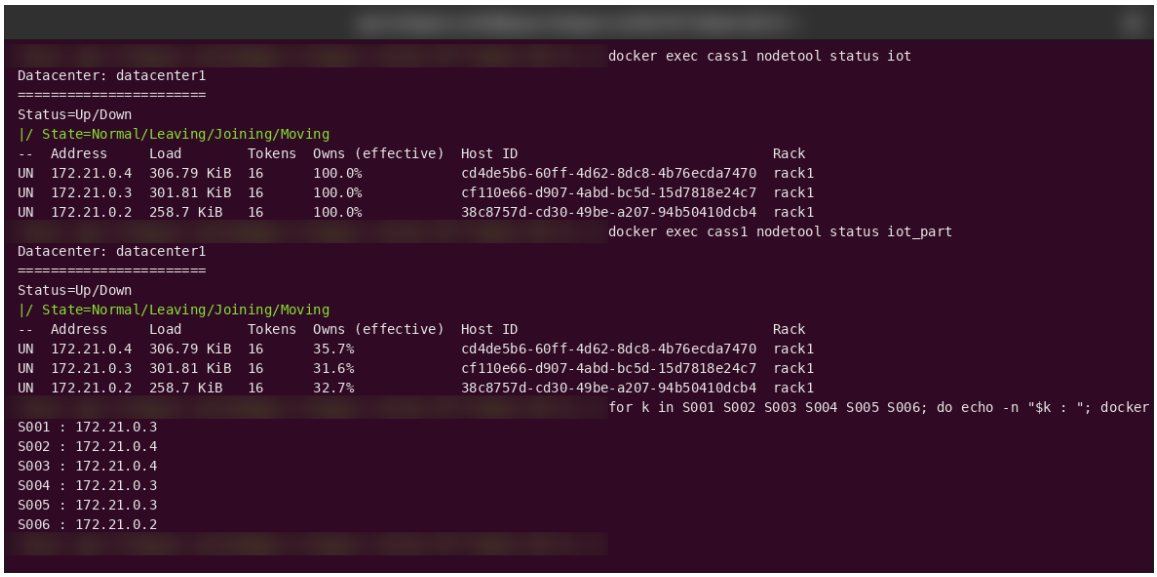


FIGURE 10 – Réplication et partitionnement : `nodetool status` sur `iot` (RF = 3) et `iot_part` (RF = 1), puis nœud responsable de chaque capteur.

Avec RF = 1, les capteurs sont répartis entre les nœuds en fonction du hachage de leur identifiant ; l'ordre d'arrivée ou le contenu des données n'y joue aucun rôle. C'est le mécanisme décrit en section 4.1, rendu visible.

Nous avons aussi testé la tolérance aux pannes sur le keyspace `iot`. Le conteneur `cass3` a été arrêté (`docker stop cass3`), puis la requête Q1 a été relancée depuis `cass1` en `QUORUM`. Avec RF = 3, deux copies restaient disponibles et la requête a réussi (figure 11). Un second arrêt ferait échouer les requêtes en `QUORUM`, puisqu'il ne resterait qu'une copie pour deux attendues, alors que celles en `ONE` aboutiraient encore : le compromis entre cohérence et disponibilité se lit directement dans le niveau choisi.

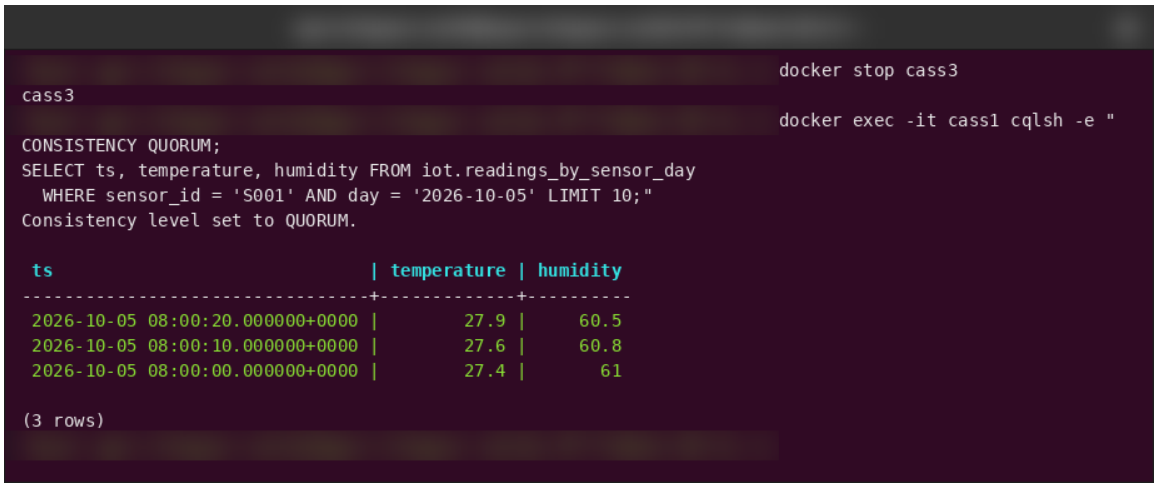


FIGURE 11 – Tolérance aux pannes : arrêt du nœud `cass3`, puis réussite de la requête Q1 en cohérence `QUORUM`.

8.11 Bilan de l'étude de cas

Les six requêtes ont conduit à quatre tables distinctes, la duplication est donc normale dans Cassandra. La clé de partition `(sensor_id, day)` borne la taille des partitions (environ 8 640 lignes par jour) et répartit la charge sur le cluster, tandis que la clé de clustering `ts` trie les mesures et rend les plages temporelles rapides. Sur le cluster, avec RF = 3, chaque nœud possède 100 % des données du keyspace `iot` ; avec RF = 1, chaque capteur est confié au nœud que désigne le hachage de son identifiant. L'arrêt de `cass3` n'a pas interrompu le service en `QUORUM`. En revanche, les requêtes sans clé de partition sont refusées, et les analyses portant sur l'ensemble des capteurs demandent un traitement extérieur à la base. La conception d'une base Wide-Column relève ainsi de l'ingénierie des accès autant que de la modélisation des données : chaque table répond à une question précise, définie avant la première écriture.

9 Performances sur un historique de dix ans : Cassandra face au relationnel

9.1 Objectif

L'étude de cas a été menée sur quelques lignes : elle montre la logique du modèle, pas son comportement quand les données s'accumulent. Nous avons donc chargé un historique de mesures croissant, d'un an jusqu'à dix ans, et mesuré à chaque étape le temps des requêtes de la section 8. Nous voulions savoir si le temps de réponse augmente avec le volume, si une mesure ancienne est plus longue à retrouver qu'une mesure récente, et comment se comporte une base relationnelle chargée des mêmes données.

9.2 Protocole

Le réseau simulé compte 50 capteurs répartis dans 5 zones. Chaque capteur envoie une mesure toutes les 15 minutes, soit 96 mesures par jour, et chaque mesure porte trois valeurs : température, humidité et niveau de batterie. Le réseau produit ainsi 4 800 mesures par jour et 1 752 000 par an. Les températures suivent un cycle journalier et saisonnier, avec de rares pics au-dessus de 35 °C. Les données sont générées de façon reproductible (graine fixe), si bien que les deux bases reçoivent exactement les mêmes valeurs. La fréquence de 15 minutes, au lieu des 10 secondes de l'énoncé, garde le volume compatible avec un poste de travail : dix ans à 10 secondes représenteraient plus d'un milliard de lignes.

Les années ont été chargées en remontant le temps : d'abord la plus récente (du 15/10/2025 au 14/10/2026), puis la précédente, et ainsi de suite jusqu'au 15/10/2016. Les mesures ont été prises après 1, 2, 3, 5 et 10 ans d'historique ; à 10 ans, chaque base contient 17 529 600 mesures. La « journée récente » est toujours le 14/10/2026, la « journée ancienne » le premier jour de l'historique (le 15/10/2016 pour dix ans).

Les essais tournent sur un même poste Linux, dans deux conteneurs Docker lancés côte à côte : un nœud Cassandra 5.0.9 (tas Java de 2 Go, keyspace `iot_bench` avec RF = 1) et un serveur PostgreSQL 15.19 (`shared_buffers` de 512 Mo). Les deux bases ne s'exécutent jamais en même temps pendant une mesure.

Code 10 – Conteneurs du banc d'essai

```
docker run -d --name cass-bench -p 9043:9042 \
  -e MAX_HEAP_SIZE=2G -e HEAP_NEWSIZE=400M cassandra:5.0
docker run -d --name pg-bench -p 5433:5432 \
  -e POSTGRES_PASSWORD=bench -e POSTGRES_DB=iot postgres:15-alpine \
```

```
  -c shared_buffers=512MB -c work_mem=64MB
```

Les temps sont mesurés depuis un programme Python (pilotes `cassandra-driver` et `psycopg2`), du départ de la requête à la réception de la dernière ligne. Chaque requête est d'abord exécutée 5 fois sans être comptée, puis 50 fois ; nous retenons la médiane. Pour les lectures « à froid », les deux serveurs sont redémarrés et le cache disque du système est vidé ; on interroge ensuite 20 couples (capteur, jour) tirés au hasard et jamais lus auparavant, une fois chacun. Avant chaque série, Cassandra écrit ses memtables sur disque (`nodetool flush`) et PostgreSQL met à jour ses statistiques (`VACUUM ANALYZE`). Les scripts et les résultats bruts se trouvent dans le dossier `benchmark/` qui accompagne ce rapport (`bench.py`, `resultats.json`).

9.3 Le modèle relationnel de comparaison

Côté Cassandra, le schéma est celui de la section 8, sans durée de vie (TTL), puisque l'historique doit être conservé. Côté PostgreSQL, nous avons écrit le schéma qu'un concepteur relationnel choisirait naturellement : une table de référence des capteurs et une table des mesures, normalisées, sans duplication. La clé primaire `(sensor_id, ts)` crée l'index B-tree qui sert la plupart des requêtes.

Code 11 – Schéma relationnel (PostgreSQL)

```
CREATE TABLE sensors (
  sensor_id  varchar(8)  PRIMARY KEY,
  zone       varchar(20) NOT NULL
);
CREATE TABLE readings (
  sensor_id    varchar(8)  NOT NULL REFERENCES sensors(sensor_id),
  ts           timestamptz NOT NULL,
  temperature  double precision,
  humidity     double precision,
  battery_pct  smallint,
  PRIMARY KEY (sensor_id, ts)
);
```

Cassandra utilise quatre tables qui dupliquent les mesures, une par type de requête. PostgreSQL n'en a que deux et calcule chaque réponse au moment de la lecture, par l'index, une jointure ou une agrégation.

9.4 Les requêtes comparées

Chaque question métier est posée aux deux bases, dans leur propre langage. Le code ci-dessous en donne la forme pour la journée la plus ancienne.

Code 12 – Requêtes du banc d'essai, version Cassandra (CQL)

```
// Q1 : 10 dernieres mesures d'un capteur pour un jour
SELECT ts, temperature, humidity FROM readings_by_sensor_day
  WHERE sensor_id = 'S017' AND day = '2016-10-15' LIMIT 10;
// Q2 : mesures entre 8 h et 12 h
SELECT ts, temperature FROM readings_by_sensor_day
  WHERE sensor_id = 'S017' AND day = '2016-10-15'
    AND ts >= '2016-10-15 08:00:00+0000' AND ts < '2016-10-15 12:00:00+0000';
// Q3 : derniere mesure de chaque capteur
SELECT * FROM last_reading_by_sensor;
// Q4 : alertes d'une zone pour un jour
```

```
SELECT * FROM alerts_by_zone_day WHERE zone = 'Zone-Est' AND day = '2026-10-14';
// Q5 : indicateurs d'un jour
SELECT COUNT(*), MIN(temperature), MAX(temperature), AVG(temperature)
  FROM readings_by_sensor_day WHERE sensor_id = 'S017' AND day = '2016-10-15';
// Q6 : indicateurs journaliers sur 365 jours (table precalculee)
SELECT day, nb_mesures, temp_min, temp_max, temp_moy FROM daily_stats_by_sensor
  WHERE sensor_id = 'S017' AND day >= '2025-10-15' AND day <= '2026-10-14';
// Q7 : requete non prevue, tous capteurs, tout l'historique
SELECT COUNT(*) FROM readings_by_sensor_day WHERE temperature > 35 ALLOW FILTERING;
```

Code 13 – Les mêmes requêtes, version relationnelle (SQL)

```
-- Q1
SELECT ts, temperature, humidity FROM readings
  WHERE sensor_id = 'S017' AND ts >= '2016-10-15' AND ts < '2016-10-16'
  ORDER BY ts DESC LIMIT 10;
-- Q2
SELECT ts, temperature FROM readings
  WHERE sensor_id = 'S017' AND ts >= '2016-10-15 08:00' AND ts < '2016-10-15 12:00
    '
  ORDER BY ts DESC;
-- Q3 : une sous-requete par capteur (LATERAL)
SELECT s.sensor_id, r.ts, r.temperature, r.humidity FROM sensors s
  CROSS JOIN LATERAL (SELECT ts, temperature, humidity FROM readings
    WHERE sensor_id = s.sensor_id ORDER BY ts DESC LIMIT 1) r;
-- Q4 : jointure avec la table des capteurs
SELECT r.ts, r.sensor_id, r.temperature FROM readings r
  JOIN sensors s ON s.sensor_id = r.sensor_id
  WHERE s.zone = 'Zone-Est' AND r.ts >= '2026-10-14' AND r.ts < '2026-10-15'
    AND r.temperature > 35 ORDER BY r.ts DESC;
-- Q5
SELECT count(*), min(temperature), max(temperature), avg(temperature) FROM
    readings
  WHERE sensor_id = 'S017' AND ts >= '2016-10-15' AND ts < '2016-10-16';
-- Q6 : agregation a la lecture, sans table precalculee
SELECT ts::date AS day, count(*), min(temperature), max(temperature),
       round(avg(temperature)::numeric, 2)
  FROM readings WHERE sensor_id = 'S017' AND ts >= '2025-10-15' AND ts < '
    2026-10-15'
  GROUP BY 1 ORDER BY 1 DESC;
-- Q7
SELECT count(*) FROM readings WHERE temperature > 35;
```

9.5 Volume stocké et chargement

La figure 12 vérifie le contenu des deux bases à la fin de l'expérience : 17 529 600 mesures du 15/10/2016 au 14/10/2026 dans PostgreSQL, et environ 180 000 partitions (capteur, jour) dans Cassandra, soit 50 capteurs $\times$ 3 652 jours.

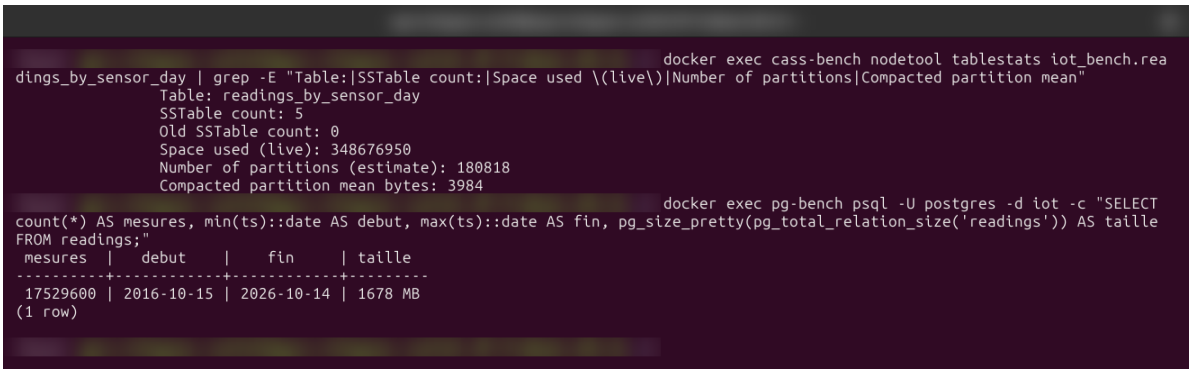


FIGURE 12 – Contenu des deux bases après dix ans de chargement : statistiques de la table des mesures dans Cassandra (`nodetool tablestats`), puis nombre de mesures, période couverte et taille de la table dans PostgreSQL.

Historique	1 an	2 ans	3 ans	5 ans	10 ans
Mesures stockées (millions)	1,75	3,50	5,26	8,76	17,53
Cassandra, table des mesures (Mo)	35	69	104	174	349
PostgreSQL, table + index (Mo)	175	350	526	876	1 755
dont index de clé primaire (Mo)	55	111	167	278	557

Pour les mêmes mesures, Cassandra occupe cinq fois moins de place. Ses SSTables sont compressées par défaut (LZ4), et la clé de partition (capteur, jour) n'est écrite qu'une fois pour ses 96 lignes, alors que PostgreSQL répète l'identifiant du capteur dans chaque ligne et chaque entrée d'index.

Le chargement d'une année a pris environ 5 minutes dans Cassandra (5 500 lignes par seconde, 51 minutes pour dix ans) et 30 à 60 secondes dans PostgreSQL (8,6 minutes pour dix ans). Ces durées ne permettent pas de comparer les moteurs, car PostgreSQL a reçu les données par `COPY`, sa commande d'import en masse, tandis que Cassandra a reçu une requête `INSERT` par mesure (quatre tables à alimenter), envoyée depuis un client Python qui saturait le processeur avant le serveur. Le test d'écriture unitaire, plus loin, compare les deux bases dans des conditions identiques.

9.6 Temps de lecture selon le volume

Le tableau suivant donne les temps médians, en millisecondes, avec un historique d'un an puis de dix ans.

	1 an (1,75 M)		10 ans (17,5 M)	
Requête (médiane, ms)	Cassandra	PostgreSQL	Cassandra	PostgreSQL
Q1, journée récente	3,69	0,34	4,71	0,44
Q1, journée la plus ancienne	3,10	0,37	3,04	1,36
Q2, plage horaire, journée ancienne	3,46	0,32	4,49	0,40
Q3, dernière mesure des 50 capteurs	5,16	1,10	7,28	2,49
Q4, alertes d'une zone pour un jour	2,29	1,97	2,53	1,11
Q5, indicateurs d'une journée ancienne	4,73	0,77	4,79	1,64
Q6, indicateurs journaliers sur 365 jours	7,75	18,84	6,97	22,28

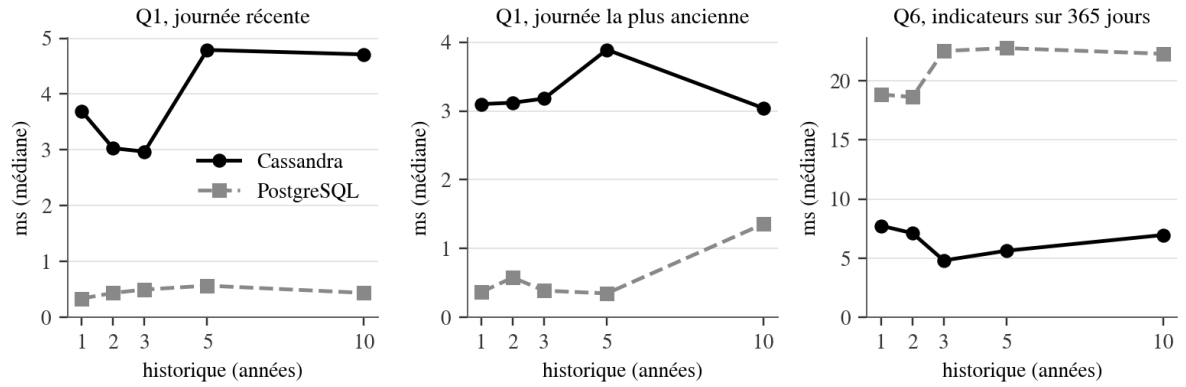


FIGURE 13 – Temps médian de Q1 (journée récente et journée la plus ancienne) et de Q6 selon la profondeur de l'historique.

Entre un et dix ans d'historique, soit dix fois plus de données, Q1 reste chez Cassandra entre 3 et 5 ms, que la journée soit récente ou vieille de dix ans. Le partitionnement l'explique : le hachage de la clé (capteur, jour) désigne la partition, qui contient toujours 96 lignes, quelle que soit la taille de la table. Ajouter des années ajoute des partitions, sans allonger celles qui existent.

Sur une seule machine, PostgreSQL est pourtant plus rapide pour les lectures par clé. Avec son index, Q1, Q2 et Q5 répondent en moins de 2 ms, quatre à dix fois plus vite que Cassandra. Le temps de PostgreSQL augmente toutefois avec le volume : Q1 sur la journée ancienne passe de 0,37 à 1,36 ms, Q5 de 0,77 à 1,64 ms et Q3 de 1,10 à 2,49 ms. L'index grossit (557 Mo à dix ans) et une part croissante de ses pages ne tient plus en mémoire. Côté Cassandra, la trace de la figure 14 indique environ 4 ms dans le serveur pour cette requête. Ce temps se répartit entre l'analyse de la requête, le coordinateur, la consultation des SSTables et la fusion des résultats ; la recherche de la partition elle-même est rapide.

Quand la question est connue d'avance, la modélisation par requête reprend l'avantage. Pour Q6, Cassandra lit 365 lignes déjà calculées dans une seule partition (7 ms), tandis que PostgreSQL agrège à chaque appel 35 040 mesures (22 ms, trois fois plus). Pour Q4, Cassandra lit une table d'alertes dédiée, en temps constant ; PostgreSQL doit joindre la table des capteurs et filtrer les mesures du jour. Il reste rapide grâce à l'index, mais son travail dépend du nombre de capteurs de la zone.

9.7 Aller chercher une donnée ancienne

Pour voir comment Cassandra retrouve une journée ancienne, nous avons activé le traçage sur Q1 pour le 15/10/2016, premier jour de l'historique (figure 14). Le nœud ne parcourt pas l'historique : grâce aux filtres de Bloom, il écarte 3 des 4 SSTables candidates, qui ne contiennent certainement pas la partition demandée, et ne lit qu'un seul fichier. La requête se termine en 3 901 µs, soit environ 4 ms, dont 1,7 ms pour l'analyse et la préparation de la requête.

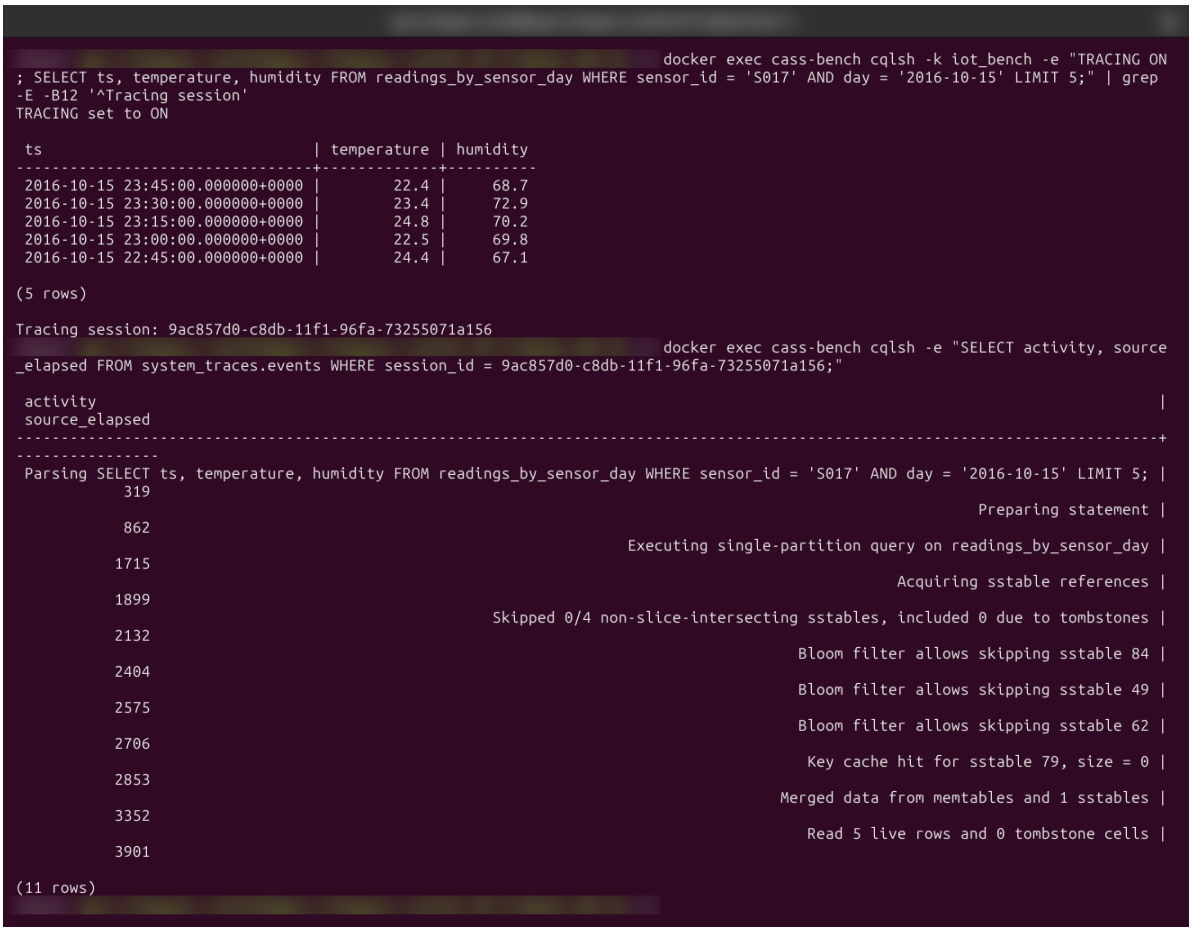


FIGURE 14 – Q1 sur la journée la plus ancienne (15/10/2016) dans Cassandra, avec la trace de la requête lue dans `system_traces.events` (temps en microsecondes depuis la réception de la requête).

Dans PostgreSQL, le plan d'exécution (figure 15) montre un parcours de l'index de clé primaire à rebours (*Index Scan Backward*), qui lit 5 pages de la mémoire partagée et s'exécute en 0,07 ms dans le serveur.

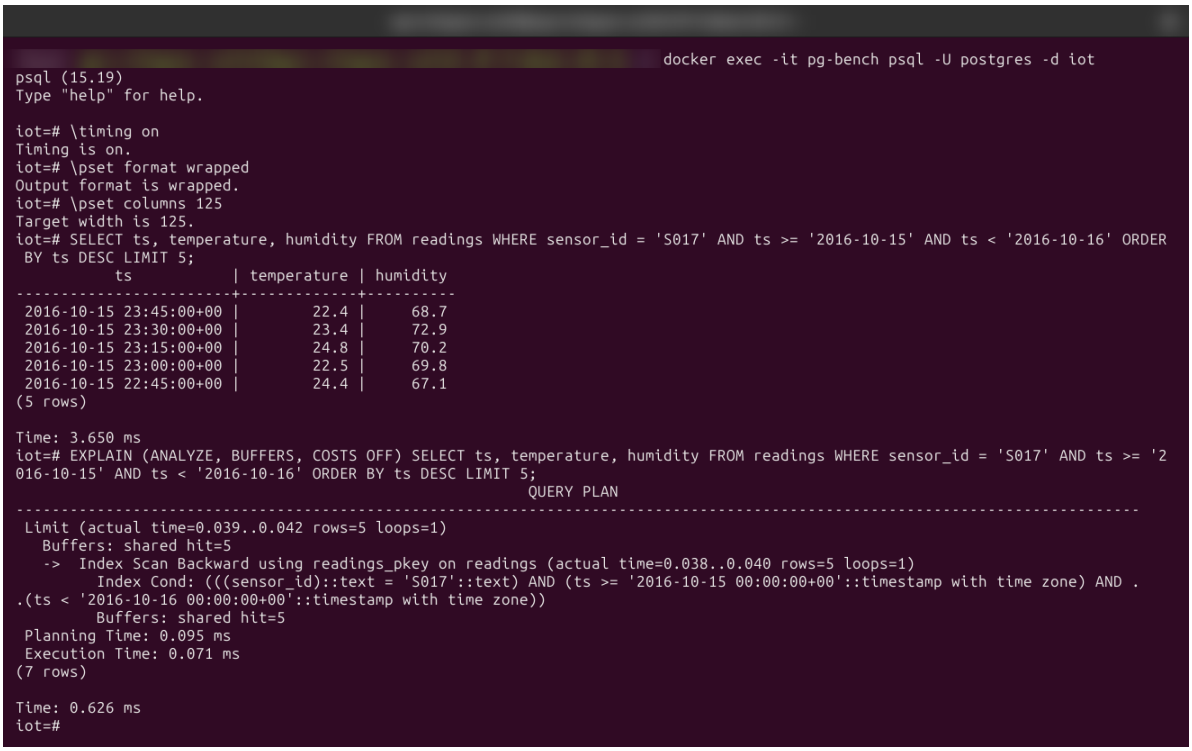


FIGURE 15 – La même requête dans PostgreSQL : résultat identique, temps mesuré par `psql`, puis plan d'exécution (`EXPLAIN ANALYZE`) qui montre l'utilisation de l'index.

Les deux bases renvoient les mêmes mesures. Les choses changent lorsque les données ne sont plus en mémoire : le tableau suivant donne le temps médian d'une première lecture, après redémarrage des serveurs et vidage du cache disque, pour 20 couples (capteur, jour) choisis au hasard, d'abord parmi les 30 derniers jours, puis dans la première année de l'historique.

Lecture à froid (médiane, ms)		1 an	2 ans	3 ans	5 ans	10 ans
Journées récentes	Cassandra	40,6	13,6	37,7	55,1	15,9
	PostgreSQL	41,7	43,0	55,9	169,0	43,3
Journées anciennes	Cassandra	39,3	17,1	39,1	53,6	15,1
	PostgreSQL	40,0	36,1	48,7	172,5	25,8

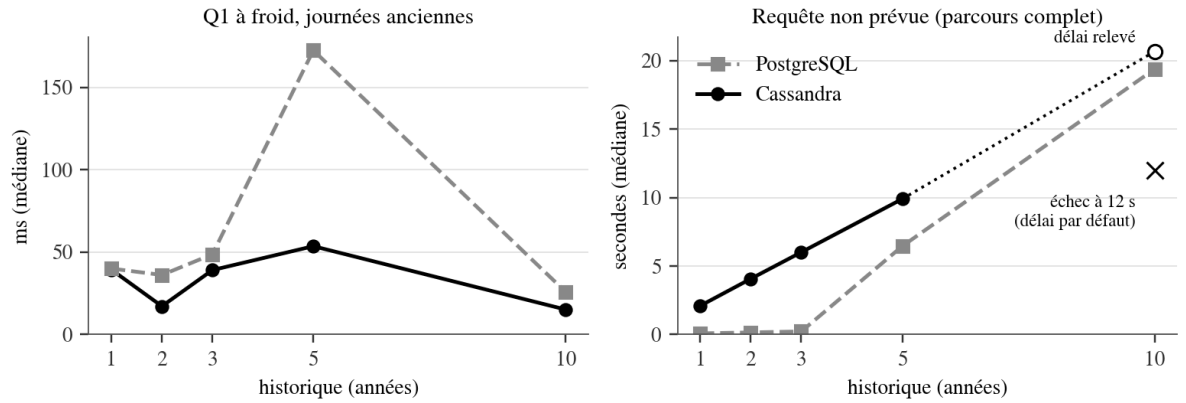


FIGURE 16 – À gauche, première lecture d'une journée ancienne après vidage des caches. À droite, durée de la requête non prévue Q7 ; à 10 ans, Cassandra échoue avec son délai par défaut (croix) et termine en 20,7 s une fois ce délai relevé (cercle).

Ces mesures sont plus dispersées que les précédentes : chaque lecture à froid dépend de l'emplacement des données sur le disque et de ce que le système garde en cache. Dans les deux bases, une journée vieille de dix ans ne coûte pas plus cher qu'une journée de la semaine, puisque l'accès passe par la clé et non par la date d'écriture. À chaque volume, Cassandra a été aussi rapide ou plus rapide que PostgreSQL à froid. Une partition (capteur, jour) est rangée d'un seul tenant dans un fichier compressé : une fois le fichier repéré par les filtres de Bloom et l'index de partition, ses 96 lignes se lisent en un accès. PostgreSQL doit d'abord descendre l'index, puis lire les pages de la table où se trouvent les lignes ; avec 5 ans d'historique, ces pages ne tenaient plus en cache et le temps médian a atteint 170 ms.

9.8 Requête non prévue sur tout l'historique

Q7 compte les mesures supérieures à 35 °C, tous capteurs confondus, sur tout l'historique. Aucune des deux bases ne dispose d'un index adapté : il faut lire toutes les mesures.

Q7 (médiane de 3 exécutions)	1 an	2 ans	3 ans	5 ans	10 ans
Mesures trouvées	1 652	3 180	4 722	7 923	15 699
Cassandra (`ALLOW FILTERING`)	2,1 s	4,1 s	6,0 s	9,9 s	échec (12 s)
PostgreSQL (parcours séquentiel)	0,08 s	0,15 s	0,22 s	6,4 s	19,4 s

Le temps de Cassandra croît linéairement avec le volume, environ 2 s par année d'historique. À dix ans, la requête échoue systématiquement (six tentatives sur six) : le serveur l'interrompt au bout de 12 s avec une erreur `ReadTimeout`, alors que le client autorisait 600 s (figure 17). En relevant ce délai dans la configuration du nœud (`native_transport_timeout`), la requête aboutit en 20,7 s et trouve 15 699 mesures, comme PostgreSQL. La section 8.9 l'annonçait : `ALLOW FILTERING` passe sur quelques lignes de test, puis devient inutilisable quand la base grandit. Dans un cluster réparti, le parcours serait partagé entre les nœuds, mais il resterait proportionnel au volume total.

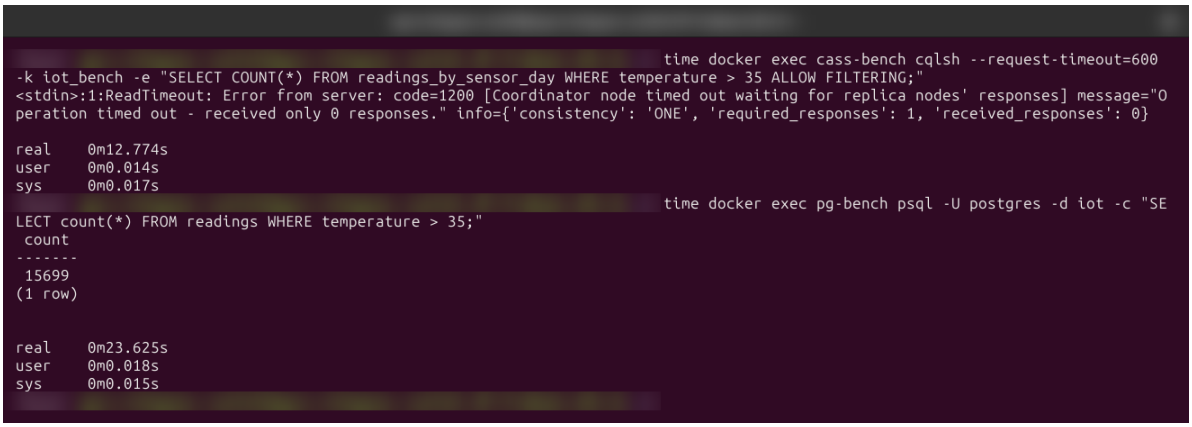


FIGURE 17 – Requête non prévue sur dix ans d'historique : Cassandra l'interrompt au bout de 12 s (`ReadTimeout`), PostgreSQL la termine en 23,6 s lors de cette exécution.

PostgreSQL termine toujours la requête. Sa courbe montre cependant un saut entre 3 et 5 ans : tant que la table (moins de 530 Mo) tient dans la mémoire du système, le parcours prend moins d'un quart de seconde ; au-delà, il faut relire la table depuis le disque et le temps est multiplié par trente. La base relationnelle accepte la question imprévue, mais sur un gros volume elle la paie aussi au prix fort.

9.9 Écritures unitaires

Dans un réseau de capteurs, chaque mesure arrive séparément. Pour comparer les deux bases dans ces conditions, 50 000 mesures d'un capteur de test ont été écrites une par une dans une base contenant déjà dix ans d'historique : 64 requêtes simultanées pour Cassandra, 8 connexions en validation automatique pour PostgreSQL. Les lignes de test ont été supprimées ensuite.

50 000 écritures unitaires	Durée	Débit
Cassandra (`INSERT`, une mesure par requête)	20,0 s	2 504 mesures/s
PostgreSQL (`INSERT`, une transaction par mesure)	164,0 s	305 mesures/s

Cassandra écrit huit fois plus vite, ce qui s'explique par le chemin d'écriture décrit en section 4.5 : une écriture Cassandra est un ajout au commit log et à la memtable, sans lecture préalable. Une insertion PostgreSQL doit mettre à jour l'index B-tree (des pages lues puis modifiées sur place), vérifier la clé étrangère et attendre la synchronisation du journal sur le disque à chaque validation.

9.10 Synthèse

Critère	Cassandra (Wide-Column)	PostgreSQL (relationnel)
Lecture par clé, données en mémoire	3 à 5 ms, constant de 1 à 10 ans	0,3 à 1,6 ms, en légère hausse avec le volume
Donnée ancienne	Même coût qu'une donnée récente	Même coût qu'une donnée récente
Lecture à froid	15 à 55 ms	26 à 172 ms
Question prévue à l'avance (Q6)	7 ms (table précalculée)	22 ms (agrégation à chaque appel)
Question imprévue (Q7, 10 ans)	Échec à 12 s ; 20,7 s avec le délai relevé	16 à 24 s
Écritures unitaires	2 504 par seconde	305 par seconde
Espace disque (10 ans)	349 Mo	1 755 Mo

Les résultats correspondent aux compromis décrits dans les sections précédentes. Cassandra garde un temps de réponse stable quelle que soit la profondeur de l'historique, écrit vite, stocke de façon compacte et répond immédiatement aux questions prévues ; une question imprévue, en revanche, échoue dès que la base grandit. PostgreSQL, bien indexé, lit plus vite sur une seule machine et répond à toutes les questions, mais ses temps augmentent avec le volume et ses écritures sont lentes.

Ces mesures ont été faites sur une seule machine. Elles ne disent donc rien de l'avantage principal de Cassandra, l'ajout de nœuds, qui multiplie la capacité d'écriture et de stockage là où PostgreSQL resterait limité à un serveur. Les temps comprennent par ailleurs le coût des pilotes Python et du réseau Docker, identique pour les deux bases, mais qui pèse davantage sur les requêtes les plus courtes. Quant à la fréquence de mesure, elle a été réduite à 15 minutes ; à 10 secondes, comme dans l'énoncé, chaque partition compterait 8 640 lignes et le volume serait multiplié par 90, ce qui accentuerait les écarts sur les écritures et sur l'espace disque.

10 Étude comparative : modèle relationnel et modèle Wide-Column

10.1 Comparaison générale

La section 3 a comparé les modèles du point de vue de leur organisation ; la section 9 les a mesurés sur le même jeu de données. Le tableau suivant rassemble les deux points de vue, critère par critère.

Critère	Modèle relationnel (PostgreSQL)	Modèle Wide-Column (Cassandra)
Point de départ de la conception	Les données, normalisées	Les requêtes, une table par requête
Schéma de l'étude de cas	2 tables, aucune duplication	4 tables, mesures dupliquées
Jointures	Exécutées à la lecture	Remplacées par la duplication à l'écriture
Transactions	ACID sur plusieurs tables	Atomicité d'un lot, transactions légères
Montée en charge	Serveur plus puissant	Ajout de nœuds
Disponibilité	Dépend du serveur principal	Aucun point de défaillance unique
Lecture par clé (mesurée)	0,3 à 1,6 ms, en hausse avec le volume	3 à 5 ms, constante de 1 à 10 ans
Écritures unitaires (mesurées)	305 par seconde	2 504 par seconde
Espace disque, 10 ans (mesuré)	1 755 Mo	349 Mo
Question imprévue (mesurée)	Toujours exécutée, 16 à 24 s	Refusée ou échouée à 12 s
Évolution des besoins	Nouvelle requête sans changer le schéma	Nouvelle table et migration de l'historique

10.2 Études de cas comparatives

Pour éprouver ces critères, nous reprenons trois applications courantes et comparons, pour chacune, la conception relationnelle et la conception Wide-Column.

Messagerie instantanée

L'application affiche les messages d'une conversation, du plus récent au plus ancien, et reçoit un flux continu d'envois. En relationnel, une table `messages` (identifiant, conversation, auteur, date, texte)

indexée sur la conversation et la date répond bien tant que le volume tient sur un serveur. En Wide-Column, la table `messages_by_conversation` a pour clé `((conversation_id, mois), date)` : chaque affichage lit une seule partition déjà triée, et les écritures se répartissent sur les nœuds. C'est le cas d'usage qui a fait naître Cassandra chez Facebook (recherche dans la boîte de réception). Dès que les utilisateurs se comptent en millions, nous retiendrions Cassandra.

Commandes d'un site marchand

Une commande décrémente le stock, enregistre le paiement et crée les lignes de commande : les trois opérations doivent réussir ensemble ou échouer ensemble, et le stock ne doit jamais devenir négatif. Le relationnel le garantit par une transaction ACID et des contraintes. En Wide-Column, il faudrait répartir ces données dans plusieurs tables sans transaction qui les englobe, et vérifier le stock par l'application ou par des transactions légères, plus lentes. Ici, nous garderions PostgreSQL. Le catalogue et l'historique de navigation du même site pourraient, eux, aller dans Cassandra.

Journaux applicatifs

Des centaines de services écrivent des lignes de journal en continu ; on les consulte par service et par heure, et on les supprime après quelques semaines. En relationnel, la table des journaux grossit sans cesse, ses index ralentissent les insertions et la purge par `DELETE` est coûteuse. En Wide-Column, la clé `((service, heure), date)` borne les partitions, les écritures en ajout seul suivent le débit et le TTL supprime les données sans requête. Cassandra convient mieux, quitte à lui adjoindre un moteur de recherche si l'on veut chercher un mot dans tous les journaux.

Cas	Exigence principale	Raison du choix	Modèle retenu
Capteurs IoT (section 9)	Écriture continue, lecture par capteur et période	Temps constant sur 10 ans, écritures 8 fois plus rapides	Wide-Column
Messagerie	Lecture d'une conversation, volume énorme	Une partition par conversation et par mois	Wide-Column
Commandes en ligne	Cohérence stricte entre stock et paiement	Transactions ACID, contraintes	Relationnel
Journaux applicatifs	Débit d'écriture, expiration	Ajout seul, TTL	Wide-Column

Au final, la taille des données compte moins que la façon dont on y accède. Des requêtes connues, beaucoup d'écritures et un besoin de distribution orientent vers le Wide-Column ; des transactions entre entités et des questions changeantes orientent vers le relationnel. Beaucoup de systèmes combinent les deux, chacun pour la partie de l'application qui lui convient.

11 Passer du modèle relationnel au modèle orienté colonnes

11.1 Est-ce possible ?

Oui, et c'est une situation courante : une application née sur une base relationnelle voit ses volumes croître jusqu'à ce qu'un serveur unique ne suffise plus. Il n'existe pas d'outil qui convertisse les tables une à une. En relationnel, les tables découlent des données (normalisation) ; en Wide-Column, elles découlent des requêtes. Il faut donc repartir de ce que l'application demande à la base, refaire le schéma, puis y recopier les données.

11.2 La démarche

La méthode suit celle de Chebotko et al. [10], déjà appliquée dans l'étude de cas.

1. On relève les requêtes réellement exécutées par l'application (code source, journal des requêtes lentes, `pg_stat_statements` sous PostgreSQL), avec leurs critères, leur ordre de tri et leur fréquence.
2. On garde le schéma entité-association, qui reste valable : il décrit les données, indépendamment de la base.
3. Chaque requête fréquente reçoit sa table, qui contient déjà le résultat d'éventuelles jointures.
4. Les critères d'égalité forment la clé de partition, les critères de plage et l'ordre de tri forment la clé de clustering. On borne la taille des partitions en ajoutant une période à la clé (*bucketing*).
5. On fixe les types, la durée de vie (TTL), la stratégie de compaction et le facteur de réplication.
6. On exporte l'historique en faisant les jointures et les agrégations côté SQL, puis on l'importe dans les tables cibles.
7. On compare le nombre de lignes et les réponses aux mêmes requêtes dans les deux bases.
8. Pendant la transition, l'application écrit dans les deux bases, ou une capture des changements (par exemple Debezium et Kafka) recopie dans Cassandra chaque modification de PostgreSQL. On bascule ensuite les lectures sur Cassandra, requête par requête.

11.3 Règles de correspondance

Le tableau suivant indique ce que devient chaque notion relationnelle.

Relationnel	Wide-Column (Cassandra)
Table normalisée	Plusieurs tables dénormalisées, une par requête
Jointure, clé étrangère	Colonnes recopiées dans la table qui en a besoin
Association 1-N	Les lignes « filles » rangées dans la partition du parent, triées par clustering
Association N-N	Deux tables, une par sens de lecture
`GROUP BY` fréquent	Table précalculée, mise à jour à l'écriture ou par un traitement périodique
Index secondaire	Table dédiée à la requête
Contrainte d'intégrité	Contrôle par l'application
Transaction sur plusieurs tables	`BATCH` logué (atomicité sans isolation) ou transaction légère (`IF NOT EXISTS`)
Identifiant auto-incrémenté	`uuid` ou `timeuuid`, générés sans coordination

11.4 Application à notre schéma

Le schéma relationnel de la section 9 (tables `sensors` et `readings`) se transforme en quatre tables Cassandra. Chaque table cible est alimentée par une requête SQL qui fait, au moment de l'export, le travail que PostgreSQL faisait à chaque lecture.

Table Cassandra	Requête SQL d'export	Requêtes servies
`readings_by_sensor_day`	Lecture de `readings`, le jour est calculé par `ts::date`	Q1, Q2, Q5
`alerts_by_zone_day`	Jointure `readings`–`sensors` pour obtenir la zone, filtre `temperature > 35`	Q4
`daily_stats_by_sensor`	`GROUP BY sensor_id, ts::date`	Q6
`last_reading_by_sensor`	Dernière mesure de chaque capteur (`LATERAL ... LIMIT 1`)	Q3

L'export utilise la commande `\copy` de `psql`, qui écrit un fichier CSV par table cible ; l'import utilise la commande `COPY` de `cqlsh`. Les scripts migrent la dernière année de l'historique, du 15/10/2025 au 14/10/2026. Les deux extraits suivants concernent la table des alertes, la seule qui exige une jointure.

Code 14 – Export depuis PostgreSQL : la jointure est faite une fois, à l'export

```
\copy (SELECT s.zone, r.ts::date,
         to_char(r.ts, 'YYYY-MM-DD HH24:MI:SS') || '+0000',
         r.sensor_id, r.temperature
       FROM readings r JOIN sensors s ON s.sensor_id = r.sensor_id
       WHERE r.temperature > 35
         AND r.ts >= '2025-10-15' AND r.ts < '2026-10-15')
  TO '/tmp/alerts.csv' WITH (FORMAT csv)
```

Code 15 – Import dans Cassandra

```
COPY iot_migre.alerts_by_zone_day (zone, day, ts, sensor_id, temperature)
  FROM '/tmp/alerts.csv';
```

Le dossier `migration/` qui accompagne ce rapport contient l'ensemble de la procédure : le schéma cible (`schema_cible.cql`), les quatre exports (`export.sql`), les quatre imports (`import.cql`), les requêtes de vérification et le script `migrer.sh` qui enchaîne les étapes entre les conteneurs du banc d'essai. La commande `COPY` de `cqlsh` convient jusqu'à quelques millions de lignes ; au-delà, on utilise DSBulk ou Spark avec le connecteur Cassandra, qui parallélisent l'import.

11.5 Ce que l'on gagne et ce que l'on perd

Après la migration, les requêtes prévues lisent une seule partition et gardent un temps constant quand l'historique s'allonge, comme l'a montré la section 9. La base peut s'étendre en ajoutant des nœuds. En contrepartie, les données sont dupliquées (une mesure au-dessus de 35 °C figure dans trois tables), la cohérence entre tables devient la responsabilité de l'application, et une nouvelle question demande une nouvelle table et une nouvelle migration de l'historique. Le passage n'est donc justifié que si les requêtes sont stables et le volume, ou le besoin de disponibilité, hors de portée d'un serveur unique.

12 Questions et éléments de réponse

Cette section propose dix questions que le thème peut susciter, avec des éléments de réponse tirés du rapport.

1. Pourquoi parle-t-on de base « orientée colonnes » alors que Cassandra range les données par partition ?

Le nom vient du modèle de données de Bigtable, où chaque valeur est repérée par une ligne, une famille de colonnes et un horodatage. Il ne décrit pas la disposition des octets sur le disque. Dans Cassandra, toutes les cellules d'une partition sont stockées ensemble. Le stockage colonnaire proprement dit (ClickHouse, Parquet) range au contraire chaque colonne séparément et vise l'analyse. Les deux familles partagent le mot « colonne », mais pas leurs usages (section 3).

2. Quelle est la différence entre clé de partition et clé de clustering ?

La clé de partition est hachée pour désigner les nœuds qui stockent la donnée ; toutes les lignes qui la partagent sont rangées ensemble. La clé de clustering trie les lignes à l'intérieur de la partition. Dans l'étude de cas, `(sensor_id, day)` regroupe les mesures d'un capteur pour une journée, et `ts` les classe de la plus récente à la plus ancienne. Une bonne clé de partition répartit la charge et garde des partitions de taille bornée.

3. Comment se passer des jointures ?

On crée une table par question, dont la clé primaire correspond aux critères de la requête, et on écrit la même donnée dans plusieurs tables. Dans notre exemple, une mesure alimente la table des mesures, celle de la dernière mesure et, si elle dépasse 35 °C, celle des alertes. La jointure est remplacée par de la duplication décidée à l'écriture. Le prix à payer est un volume plus grand et la nécessité de garder ces tables cohérentes.

4. Que se passe-t-il quand un nœud tombe en panne ?

Chaque partition existe en plusieurs copies (facteur de réplication). Avec RF = 3 et un niveau de cohérence `QUORUM`, deux copies sur trois suffisent : notre essai a montré que la requête Q1 aboutit après l'arrêt de `cass3`. Les écritures manquées par le nœud absent sont conservées par un autre nœud (*hinted handoff*) et lui sont transmises à son retour ; `nodetool repair` corrige les écarts restants.

5. Cassandra garantit-elle la cohérence des données ?

Elle laisse l'application choisir, requête par requête. Si le nombre de copies interrogées en lecture (R) et le nombre de copies confirmées en écriture (W) vérifient $R + W > RF$, la lecture voit toujours la dernière écriture. Avec un niveau plus faible, comme `ONE`, la réponse est plus rapide et le système reste disponible pendant une panne, au risque de lire une valeur un peu ancienne. C'est le compromis décrit par le théorème CAP.

6. Pourquoi les écritures sont-elles si rapides ?

Une écriture est ajoutée au commit log, journal écrit séquentiellement, puis à la memtable en mémoire ; le client est alors confirmé. Rien n'est lu ni modifié sur place : les fichiers sur disque (SSTables) sont créés plus tard et fusionnés en arrière-plan. Dans notre essai, Cassandra a enregistré 2 504 mesures par seconde, une par requête, contre 305 pour PostgreSQL, qui doit mettre à jour son index et synchroniser son journal à chaque transaction.

7. Une donnée vieille de dix ans est-elle plus longue à retrouver qu'une donnée récente ?

Non, tant que la requête fournit la clé de partition. Le hachage de (capteur, jour) désigne directement la partition, et les filtres de Bloom évitent d'ouvrir les fichiers qui ne la contiennent pas. Nous avons mesuré 3 à 5 ms pour la journée la plus ancienne comme pour la plus récente, avec un historique d'un an comme de dix ans (section 9).

8. Pourquoi `ALLOW FILTERING` est-il déconseillé ?

Sans clé de partition, Cassandra doit lire toutes les partitions de tous les nœuds pour trouver les lignes demandées. Le coût augmente avec la taille de la base. Dans notre essai, compter les mesures supérieures

à 35 °C prenait 2 s avec un an d'historique, 10 s avec cinq ans, et échouait au bout de 12 s avec dix ans. Une question fréquente doit avoir sa propre table ; une analyse ponctuelle relève d'un outil comme Spark.

9. Quand vaut-il mieux choisir une base relationnelle ?

Quand l'application a besoin de transactions sur plusieurs tables (comptabilité, virements), quand les questions ne sont pas connues d'avance, ou quand le volume tient sur un seul serveur. Nos mesures montrent qu'un PostgreSQL bien indexé lit plus vite que Cassandra sur une seule machine et répond à toutes les requêtes. Cassandra se justifie lorsque les écritures sont nombreuses, que les données doivent être réparties sur plusieurs machines ou que le service doit rester disponible malgré les pannes.

10. Cassandra, HBase ou ScyllaDB : comment choisir ?

Cassandra convient à un service qui doit rester disponible, éventuellement sur plusieurs datacenters, sans serveur maître. HBase s'impose lorsque les données vivent déjà dans l'écosystème Hadoop et qu'une cohérence forte par ligne est nécessaire, au prix d'un déploiement plus lourd (HDFS, ZooKeeper). ScyllaDB reprend l'interface de Cassandra avec un moteur écrit en C++ ; son intérêt se vérifie par un test sur la charge réelle, et sa licence doit être examinée avant un usage professionnel.

Conclusion

Les bases Wide-Column répondent à un besoin précis : stocker de très grands volumes de données qui arrivent en continu, sur des machines ordinaires, sans point de défaillance unique. Elles s'appuient pour cela sur un partitionnement par hachage, une réplication sans maître, une cohérence réglable et un moteur d'écriture en ajout seul.

Sur dix ans de mesures (section 9), le temps de lecture de Cassandra n'a pas bougé avec le volume, alors que la requête imprévue a fini par échouer.

Elles imposent en retour de connaître les requêtes avant de créer les tables, d'accepter la duplication et de renoncer aux jointures et aux transactions complexes. Le choix d'un tel système se justifie donc par l'adéquation entre les besoins d'accès et le modèle, plus que par la seule taille des données. Dans une architecture actuelle, on trouve souvent une base Wide-Column pour l'ingestion et un entrepôt colonnaire pour l'analyse, alimenté à partir d'elle.

Références

[1] F. Chang, J. Dean, S. Ghemawat et al. *Bigtable : A Distributed Storage System for Structured Data*. OSDI, 2006.

[2] G. DeCandia, D. Hastorun et al. *Dynamo : Amazon's Highly Available Key-value Store*. SOSP, 2007.

[3] A. Lakshman, P. Malik. *Cassandra : A Decentralized Structured Storage System*. ACM SIGOPS Operating Systems Review, 44(2), 2010.

[4] P. O'Neil, E. Cheng, D. Gawlick, E. O'Neil. *The Log-Structured Merge-Tree (LSM-Tree)*. Acta Informatica, 33(4), 1996.

[5] S. Gilbert, N. Lynch. *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services*. ACM SIGACT News, 33(2), 2002.

[6] D. Abadi. *Consistency Tradeoffs in Modern Distributed Database System Design*. IEEE Computer, 45(2), 2012.

[7] Apache Cassandra Documentation, `https://cassandra.apache.org/doc/latest/`.

[8] Apache HBase Reference Guide, `https://hbase.apache.org/book.html`.

[9] ScyllaDB Documentation, `https://docs.scylladb.com/`.

[10] A. Chebotko, A. Kashlev, S. Lu. *A Big Data Modeling Methodology for Apache Cassandra*. IEEE International Congress on Big Data, 2015.

[11] E. F. Codd. *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, 13(6), 1970.